Seminar
Special Operators. Usage. More Design Patterns.

1. ++ and -- operators.

- Prefix vs postfix ++/--:
- Prefix (++v): modify, then return **reference to self**.

```
Vector2D& operator++() {
    ++x;
    ++y;
    return *this;
}
```

- Postfix (v++): save the old value, increment, return **old value by value**.

```
Vector2D  operator++(int) {  // dummy int = postfix
    Vector2D old = *this;
    ++(*this);  // reuse prefix
    return old;
}
```

- **Rule:** always implement postfix in terms of prefix. The dummy `int` parameter has no name and is never used - it is just C++'s way of distinguishing between the two signatures.

2. Assignment operator.

- If your class manages a resource (heap memory, file handle, etc.), you **must** write your own `operator=`.
- **Self-assignment guard** is critical - without it, you delete your own data before copying it:

```
MyClass& operator=(const MyClass& other) {
    if (this == &other)
        return *this;
    // code...
    return *this;
}
```

- **Rule of Three:** if you write `operator=`, you almost certainly need a **copy constructor** and a **destructor** too - they all manage the same resource.
- Copy-and-swap idiom - gives strong exception safety for free:

```
MyClass& operator=(MyClass other) {
    swap(*this, other);  // swap is noexcept
    return *this;
   // old data freed when 'other' goes out of scope
}
```

3. Stream operators << and >>.

- Must be **free functions** — you cannot add a member to `std::ostream`.
- Return the stream **by reference** to allow chaining: `cout << a << b << c;`

- Declare as `friend` inside the class if private data is needed.

```
std::ostream& operator<<(std::ostream& os, const Vector2D& v)
{
    return os << "(" << v.x << ", " << v.y << ")";
}

std::istream& operator>>(std::istream& is, Vector2D& v) {
    return is >> v.x >> v.y;
}
```

4. operator[] - subscript / indexing.

- Always provide **two versions**: a non-const one that returns a reference (allows writes), and a const one that only allows reads.

```
class Array {
    double data[42];
public:
    double& operator[](int i)       { return data[i]; }
    double  operator[](int i) const { return data[i]; }
};
```

5. operator() - callable objects (functors).

- Overloading `operator()` makes an object **callable** - it can be used like a function. These are called **functors**.

```
class Multiplier {
public:
    Multiplier(int f) : factor(f) {}
    int operator()(int x) const { return x * factor; }
private:
    int factor;
};

Multiplier triple(3);
std::cout << triple(7);  // prints 21
```

6. operator* and operator->.

- These two operators let a class **wrap a pointer** while acting transparent to the caller.

```
template <typename T>
class ScopedPtr {
public:
    explicit ScopedPtr(T* p) : ptr(p) {}
    ~ScopedPtr() { delete ptr; }
    T& operator*()  const { return *ptr; }
    T* operator->() const { return ptr; }
```

```
    explicit operator bool() const { return ptr != nullptr; }
private:
    T* ptr;
};

ScopedPtr<Person> p(new Person("Alice"));
p->greet();   // calls Person::greet through the pointer
(*p).greet();  // same thing
```

- `operator->` must return either a raw pointer or an object that **also** has `operator->` - the compiler chains calls until it hits a raw pointer.

7. Design Patterns.

- **Fluent Interface / Method Chaining** - `operator+=` returning `*this` by reference enables chaining.
  `operator<<` on streams is the canonical example.

- **Proxy Pattern** - `operator[]` can return a **proxy object** instead of a plain reference, allowing it to distinguish reads from writes.
  Classic use: a `BitVector` that packs 8 booleans into one byte (8 bits).

```
class BitVector {
public:
    class BitProxy {
    public:
        BitProxy(unsigned char& b, int bit)
            : byte(b), bit(bit) {}

        // write
        BitProxy& operator=(bool val) {
            if (val)
                byte |= (1 << bit);
            else
                byte &= ~(1 << bit);
            return *this;
        }

        // read
        operator bool() const { return (byte >> bit) & 1; }
    private:
        uint8_t& byte;
        int bit;
    };

    BitProxy operator[](int i) {
        return BitProxy(data[i/8], i % 8);
```

```
    }

private:
    unsigned char* data;
};

BitVector bv;
bv[3] = true;    // calls BitProxy::operator=(bool) - write
bool x = bv[3];  // calls BitProxy::operator bool() - read
```

- **Iterator Pattern** - overloading **++**, *****, and **!=** is all that is needed to make an object work with range-for loops.

```
class Range {
public:
    Range(int from, int to) : current(from), end(to) {}
    Range& begin() { return *this; }
    Range  end()   { return Range(end, end); }
    bool operator!=(const Range& other) const {
        return current != other.current;
    }
    Range& operator++() { ++current; return *this; }
    int operator*() const { return current; }

private:
    int current;
    int end;
};

for (int i : Range(1, 5))
    std::cout << i << " ";
// Prints: 1 2 3 4
```

- A range-for loop expands to a loop that uses: `begin()`, `end()`, **!=**, **++**, *****. These operators are all it takes to integrate it with the language.